Course : AAD – ALGORITHM ANALYSIS AND DESIGN

Chapter 4

Design and analysis of Divide and Conquer Algorithms

Dr Rahaf Ghazal

2025-2026

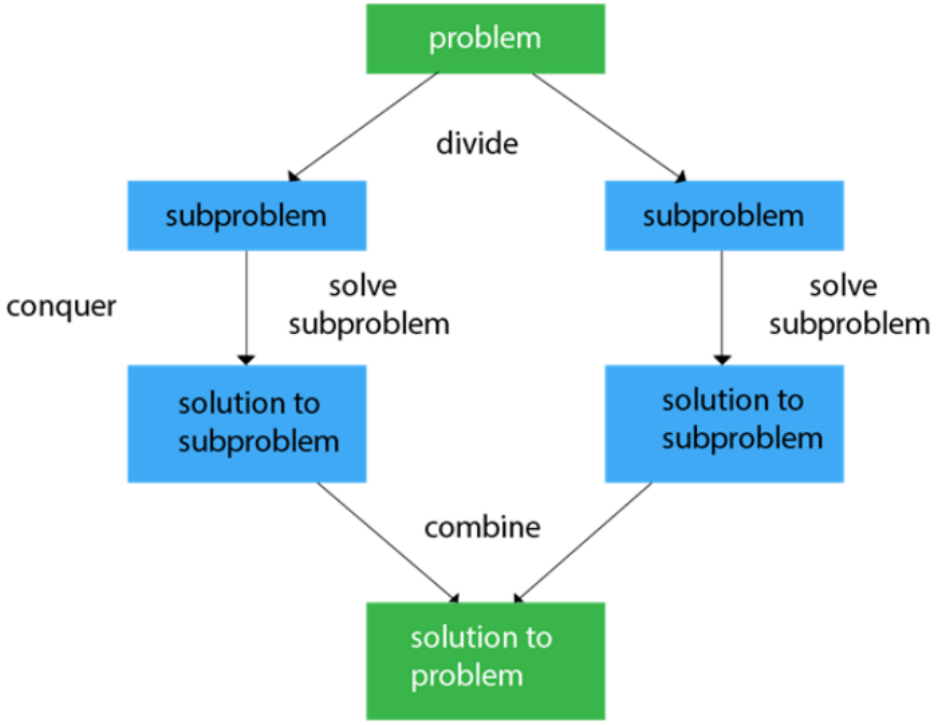
- الخوارزميات العودية (Recursive Algorithms)
- الخوارزميات التتبع الخلفي - التراجعية (Backtracking algorithms)
- خوارزميات البرمجة الديناميكية (Dynamic Programming Algorithms)
- خوارزميات التفريق ثم الحل (Divide and Conquer Algorithms) ⟸
- الخوارزميات الجشعة والأنانية (Greedy Algorithms)

Introduction

- In this approach ,we solve a problem recursively by applying 3 steps:
  1. **DIVIDE**-break the problem into several and independent subproblems of smaller size.
  2. **CONQUER**-solve the problem recursively.
  3. **COMBINE**-combine these solutions to create a solution to the original problem.

Note: Divide and Conquer is mainly useful when we divide a problem into independent subproblems. If we have overlapping subproblems, then we use Dynamic Programming.

Abstraction model for divide and conquer algorithm



Algorithm D and C (P)

{

if small(P)

then return S(P)

else

{ divide P into smaller instances P_1 ,P_2 .....P_k

Apply D and C to each sub problem

Return combine (D and C(P_1)+ D and C(P_2)+.....+D and C(P_k))

}

Improving Efficiency Through Divide and Conquer

- **Divide and conquer can improve algorithm efficiency**. In sorting algorithms, quick sort, merge sort, and heap sort are faster than selection, bubble, and insertion sort because they apply the divide and conquer strategy.

- This raises the question: Why can divide and conquer improve algorithm efficiency? In other words, why is dividing a large problem into multiple subproblems, solving the subproblems, and merging their solutions more efficient than directly solving the original problem? This question can be discussed from two aspects: **operation count and parallel computation.**

1. Operation Count Optimization:

Taking "bubble sort" as an example, processing an array of length **n** requires n^2 time. Suppose we divide the array into two subarrays from the midpoint, the division requires **O(n)** time, sorting each subarray requires $(\frac{n}{2})^2$ time, and merging the two subarrays requires **O(n)** time, resulting in an overall time complexity of:

$$n + 2 * \left(\frac{n}{2}\right)^2 + n = \frac{n^2}{2} + 2n$$

Next, we compute the following inequality, where the left and right sides represent the total number of operations before and after division, respectively:

$$n^2 > \frac{n^2}{2} + 2n$$

$$n^2 - \frac{n^2}{2} - 2n > 0$$

$$n(n-4) > 0$$

This means that when n > 4 , the number of operations after division is smaller, and sorting efficiency should be higher.

Going further, **what if we continuously divide the subarrays from their midpoints into two subarrays** until the subarrays have only one element? This approach is actually "merge sort", with a time complexity of **O(nlogn).**

2. Parallel Computation Optimization: We know that the subproblems generated by divide and conquer are independent of each other, so they can typically be solved in parallel.

Parallel optimization is particularly effective in multi-core or multi-processor environments, as the system can simultaneously handle multiple subproblems, making fuller use of computing resources and significantly reducing overall runtime.

Real-World Applications

- **Database and Search Engine Sorting:** Algorithms like Merge Sort and Quick Sort, which break large lists into smaller sub-lists to sort them, are used for managing and organizing data, such as sorting products by price.
- **Data Retrieval (Binary Search):** Used in GPS applications, e-commerce, and any system requiring fast data retrieval from a sorted dataset by repeatedly splitting the search space in half.
- **Parallel Computing:** Divide and conquer is inherently parallelizable, allowing massive tasks to be divided and processed simultaneously across multi-core processors or distributed systems.

Algorithms which are based on Divide and Conquer

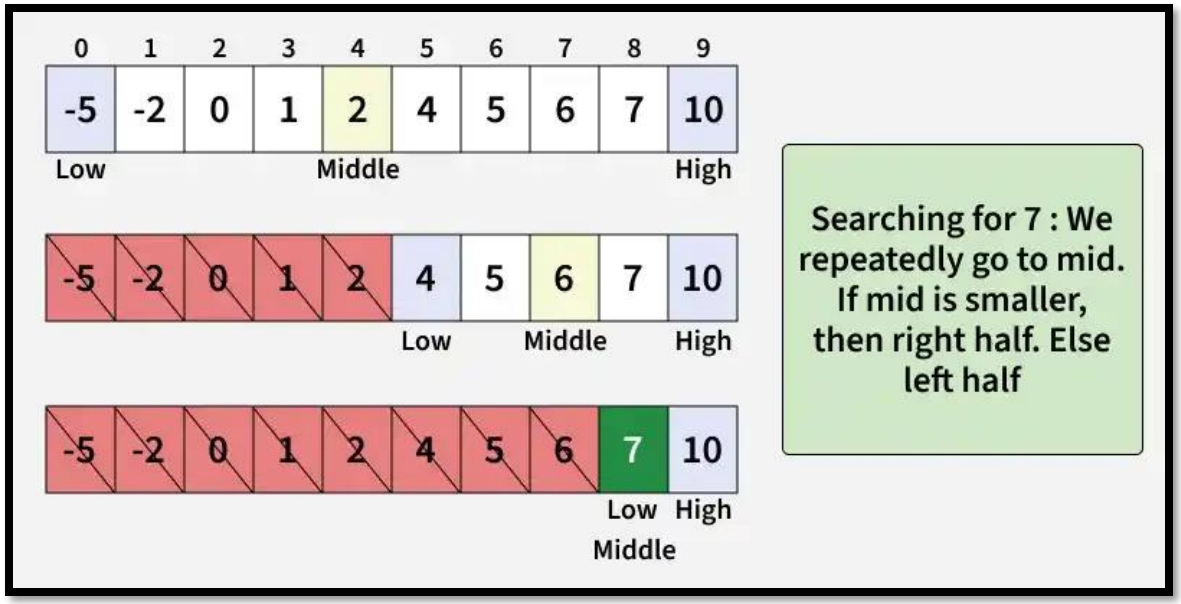
Application of Divide & Conquer in Binary Search:

Binary Search *is a search algorithm that efficiently finds a target value within a sorted array. It repeatedly divides the search space in half until the target is found or the search space is empty.*

Algorithm Steps:

- Initialize **low**, **high**, and **mid** indices.
- Compare the **target** with the element at the **mid** index.
- If the **target** is found, return the index.
- If the **target** is smaller, update the **high** index to **mid - 1.**
- If the **target** is larger, update the **low** index to **mid + 1.**
- Repeat the process until the **target** is found or the search space is empty.

```
def binary_search_recursive(arr, low, high, x):
if high >= low
    mid = (high + low) DIV 2
    if arr[mid] = x:
        return mid
    elseif arr[mid] > x:
        return binary_search_recursive(arr, low, mid - 1, x)
    else
        return binary_search_recursive(arr, mid + 1, high, x)
else
 return -1 # العنصر غير موجود
```



Recurrence for binary search

$$T(n) = 1 \quad T\left(\frac{n}{2}\right) + O(1)$$

$$T(n) = O(log(n))$$

subproblems

subproblem size

work dividing and combining

T(n): The time it takes to solve a problem of size n.

T(n/2): At each step, the binary search algorithm divides the sorted array in half. The algorithm then recursively solves the problem on only one of those halves.

O(1) or C (constant): This represents the constant amount of work performed at each step, which includes: Calculating the middle index, Comparing the target value with the middle element, and Making the decision to search the left or right sub-array, or concluding that the element is found.

Homework

Study the Best Case and Average Case Time Complexity of Binary Search Algorithm

Algorithms which are based on Divide and Conquer

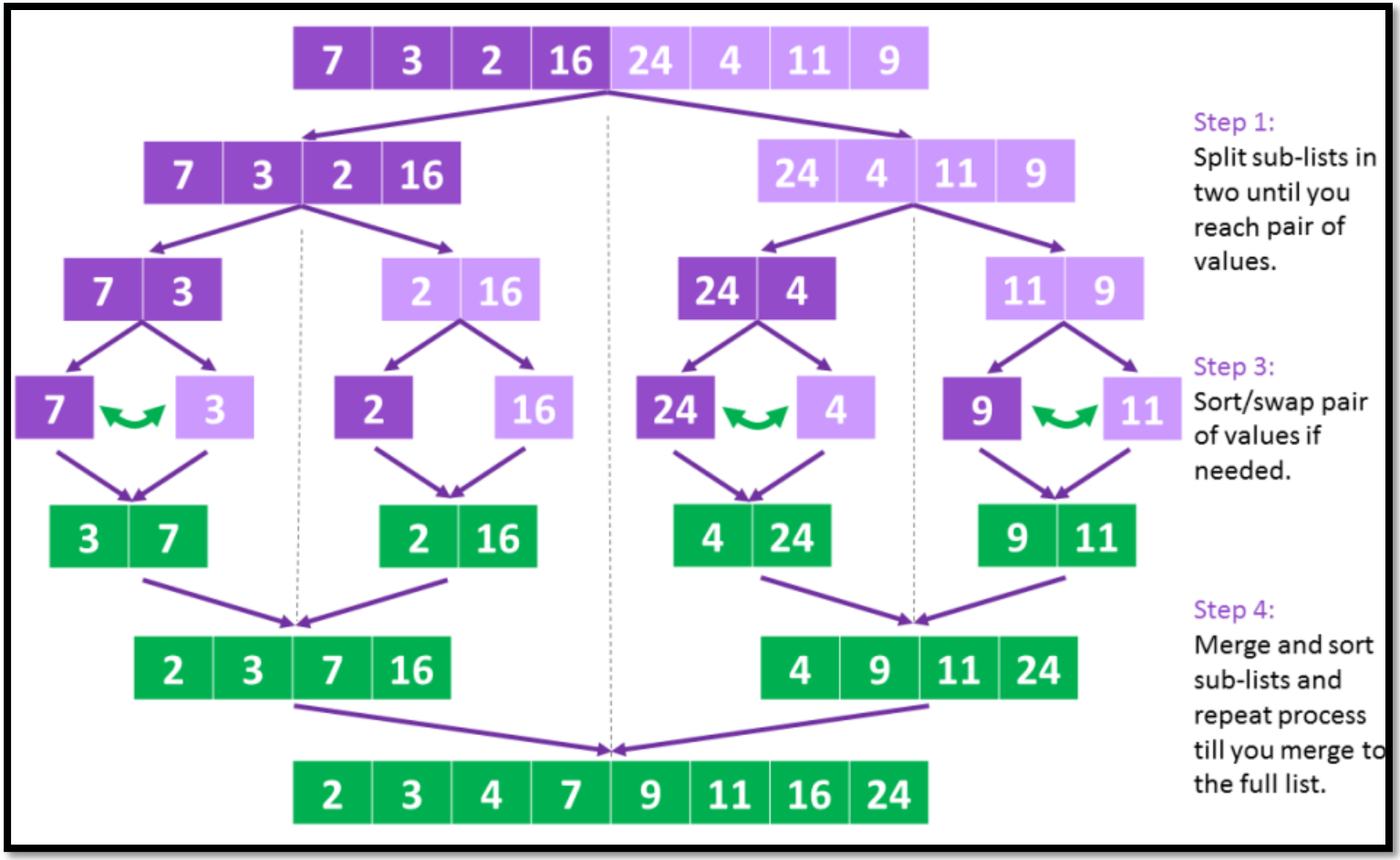
Application of Divide & Conquer in Merge Sort:

Merge Sort *is a divide-and-conquer sorting algorithm that recursively divides an array into* ***two halves****, sorts each half, and then merges them to produce a sorted array.*

Algorithm Steps:

- Divide the unsorted array into **two halve**s.
- Recursively apply Merge Sort to each half.
- Merge the sorted halves to produce a sorted array.

```
void mergeSort(int arr[],int l,int r){
    if(l<r){
    int m =l+ (r-l)/2;
    mergeSort(arr,l,m);
    mergeSort(arr,m+1,r);
    merge(arr,l,m,r);
    }
}
```



Recurrence for Merage Sort Algorithm:

$$T(n) = 2 \quad T\left(\frac{n}{2}\right) + O(n)$$

subproblems

subproblem size

work dividing and combining

T(n)=Big-O(n log n) When the array is sorted in reverse order

T(n): Represents the total time taken by the algorithm to sort an array of size n.

2T(n/2): represents time taken by the algorithm to recursively sort the two halves of the array. Since each half has n/2 elements, we have two recursive calls with input size as (n/2).

O(n): represents the time taken to merge the two sorted halves.

Homework

Study the Best Case and Average Case Time Complexity of Merge sort Algorithm

Algorithms which are based on Divide and Conquer

Example 3: matrix multiplication:

Matrix Multiplication: Given two matrices mat1[][] of size n × m and mat2[][] of size m × q, find their matrix product, where the resulting matrix has dimensions n × q. Note: we'll study the special case, when m=n. That mean :

Matrix Multiplication (MATRIX)
Input: $n \times n$ matrices of integers A and B
Output: $C = A \cdot B$ a $n \times n$ matrix.

1) Standard approach By nested loop

for $i \leftarrow 1$ **to** n
  do for $j \leftarrow 1$ **to** n
    do $c_{ij} \leftarrow 0$
      for $k \leftarrow 1$ **to** n
        do $c_{ij} \leftarrow c_{ij} + a_{ik} \cdot b_{kj}$

$$\begin{bmatrix} a_{1,1} & \cdots & a_{1,n} \\ \vdots & \ddots & \vdots \\ a_{n,1} & \cdots & a_{n,n} \end{bmatrix} \cdot \begin{bmatrix} b_{1,1} & \cdots & b_{1,n} \\ \vdots & \ddots & \vdots \\ b_{n,1} & \cdots & b_{n,n} \end{bmatrix} = \begin{bmatrix} c_{1,1} & \cdots & c_{1,n} \\ \vdots & \ddots & \vdots \\ c_{n,1} & \cdots & c_{n,n} \end{bmatrix}$$

$$c_{i,j} = \sum_{k=1}^{n} a_{i,k} \cdot b_{k,j}$$

So each entry requires n multiplications and $n - 1$ additions.
There are n^2 entries.
This is an $O(n^3)$ algorithm.

2) Divide and conquer approach *(assuming that n is a power of 2):*

الفكرة هي تقسيم المصفوفتين المربعتين A,B (كل منهما) إلى أربعة مصفوفات مربعة جزئية(أرباع)، ومن ثم استخدام مفهوم الدالة العودية على كل جزء (ربع) والحصول على النتيجة لكل ربع. أخيرا نجمع نتائج عمليات الجداء ونحصل على مصفوفة الجداء المطلوبة.

$$\begin{bmatrix} r & s \\ t & u \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

$$C = A \cdot B$$

$$\left.\begin{aligned} r &= ae + bg \\ s &= af + bh \\ t &= ce + dg \\ u &= cf + dh \end{aligned}\right\} \begin{aligned} &8 \text{ mults of } (n/2)\times(n/2) \text{ submatrices} \\ &4 \text{ adds of } (n/2)\times(n/2) \text{ submatrices} \end{aligned}$$

MMult(A, B, n)

1. If $n = 1$ Output $A \times B$
2. Else
3. Compute $A^{11}, B^{11}, \ldots, A^{22}, B^{22}$ by computing $m = n/2$
4. $X_1 \leftarrow MMult(A^{11}, B^{11}, n/2)$
5. $X_2 \leftarrow MMult(A^{12}, B^{21}, n/2)$
6. $X_3 \leftarrow MMult(A^{11}, B^{12}, n/2)$
7. $X_4 \leftarrow MMult(A^{12}, B^{22}, n/2)$
8. $X_5 \leftarrow MMult(A^{21}, B^{11}, n/2)$
9. $X_6 \leftarrow MMult(A^{22}, B^{21}, n/2)$
10. $X_7 \leftarrow MMult(A^{21}, B^{12}, n/2)$
11. $X_8 \leftarrow MMult(A^{22}, B^{22}, n/2)$
12. $C^{11} \leftarrow X_1 + X_2$
13. $C^{12} \leftarrow X_3 + X_4$
14. $C^{21} \leftarrow X_5 + X_6$
15. $C^{22} \leftarrow X_7 + X_8$
16. Output C
17. End If

Analysis of D&C algorithm

$$T(n) = 8 \quad T\left(\frac{n}{2}\right) + O(n^2)$$

submatrices

submatrix size

work adding submatrices

The Master Theorem gives us

$$T(n) = \Theta(n^{\log_2(8)}) = \Theta(n^3)$$

No better than the ordinary algorithm.

There is a method called Strassen's approach to improve the previous cost, (which is based on the principle of division and then solution), but it is a very slight improvement, where T(n) becomes of the form:

$$T(n) = \Theta(n^{\log_2(7)}) = \Theta(n^{2.8})$$

3) Strassen's approach *(assuming that n is a power of 2):*

For reading

Strassen(A, B)

1. If $n = 1$ Output $A \times B$
2. Else
3. Compute $A^{11}, B^{11}, \ldots, A^{22}, B^{22}$ % by computing $m = n/2$
4. $P_1 \leftarrow Strassen(A^{11}, B^{12} - B^{22})$
5. $P_2 \leftarrow Strassen(A^{11} + A^{12}, B^{22})$
6. $P_3 \leftarrow Strassen(A^{21} + A^{22}, B^{11})$
7. $P_4 \leftarrow Strassen(A^{22}, B^{21} - B^{11})$
8. $P_5 \leftarrow Strassen(A^{11} + A^{22}, B^{11} + B^{22})$
9. $P_6 \leftarrow Strassen(A^{12} - A^{22}, B^{21} + B^{22})$
10. $P_7 \leftarrow Strassen(A^{11} - A^{21}, B^{11} + B^{12})$
11. $C^{11} \leftarrow P_5 + P_4 - P_2 + P_6$
12. $C^{12} \leftarrow P_1 + P_2$
13. $C^{21} \leftarrow P_3 + P_4$
14. $C^{22} \leftarrow P_1 + P_5 - P_3 - P_7$
15. Output C
16. End If

Strassen's algorithm is based on the following observation:

$$C^{11} = P_5 + P_4 - P_2 + P_6 \qquad C^{12} = P_1 + P_2$$
$$C^{21} = P_3 + P_4 \qquad C^{22} = P_1 + P_5 - P_3 - P_7$$

where

$$P_1 = A^{11}(B^{12} - B^{22})$$
$$P_2 = (A^{11} + A^{12})B^{22}$$
$$P_3 = (A^{21} + A^{22})B^{11}$$
$$P_4 = A^{22}(B^{21} - B^{11})$$
$$P_5 = (A^{11} + A^{22})(B^{11} + B^{22})$$
$$P_6 = (A^{12} - A^{22})(B^{21} + B^{22})$$
$$P_7 = (A^{11} - A^{21})(B^{11} + B^{12})$$

For reading

The best current upper bound for multiplying two matrices of size $n \times n$ is $\mathcal{O}(n^{2.32})$ (by using similar idea, but instead of dividing a matrix into 4 quaters, people divide them into a bigger number of submatrices).

Open question: Can we multiply two $n \times n$ matrices in time $\mathcal{O}(n^2)$?

ملاحظة: لا يُفضّل استخدام طريقة ستراسن عمليًا في كثير من الأحيان نظرًا لارتفاع معاملاتها الثابتة، مما يجعل الطريقة البسيطة أسرع في التطبيقات الشائعة. كما أن عمليات قسمة المصفوفات الفرعية المتكررة تتطلب ذاكرة إضافية

Homework

Study the Quick sort Algorithm